2026 Operating System

Lab 2. Data Concurrency

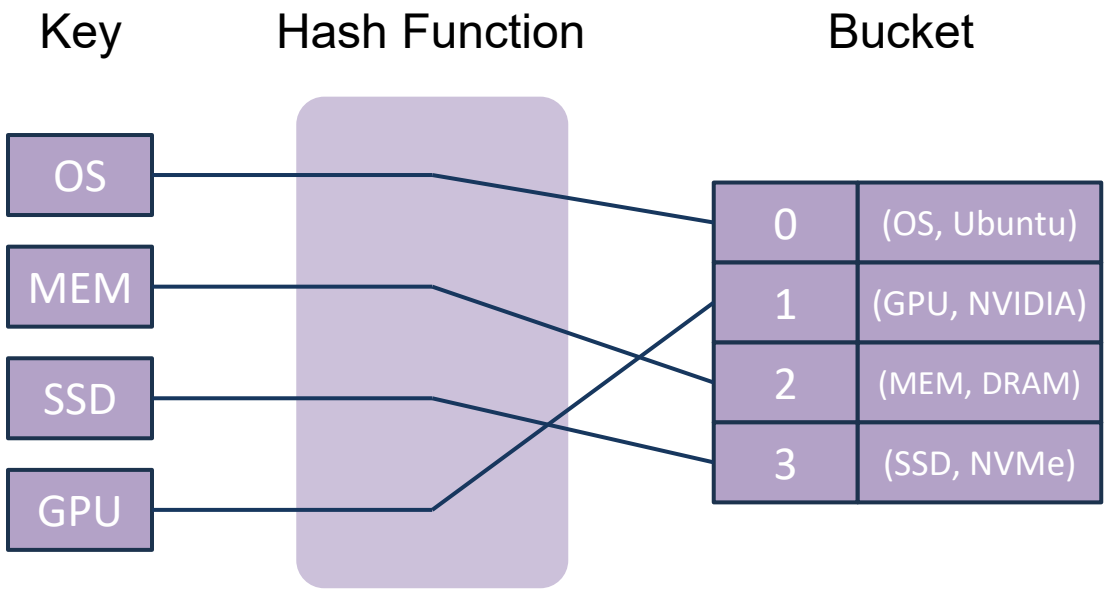
2026.04.29

T.A. 위다연

weedayeon@dankook.ac.kr

Lab2. Data Concurrency

- Implement Lock
  - Implement the following three versions of the HashTable
    - Without Lock (Original HashTable)
    - Coarse-grained Lock
    - Fine-grained Lock



Data Structure

- HashTable
  - A data structure that stores key-value pairs using a hash function
  - Maps each key to an index in an array for efficient access
  - Support fast search, insertion, and deletion
  - Average time complexity of O(1)

  - Structure
    - Stores data in buckets based on the hash value of each key
    - A hash function converts a key into an array index
    - Collisions may occur when multiple keys map to the same bucket
    - Collisions are handled using techniques such as chaining or open addressing
    - Performance depends on the quality of the hash function and load factor

Data Structure

- HashTable
  - Pros (Strengths)
    - Average O(1) time for search, insert, and delete
    - Very fast access using direct indexing via hash function
    - Simple lookup logic compared to tree-based structures
  - Cons (Weakness)
    - Performance degrades to O(N) in worst-case (due to collisions)
    - Requires a good hash function to avoid clustering
    - Handling collisions adds complexity (chaining, open addressing)
    - Memory overhead due to unused buckets and resizings

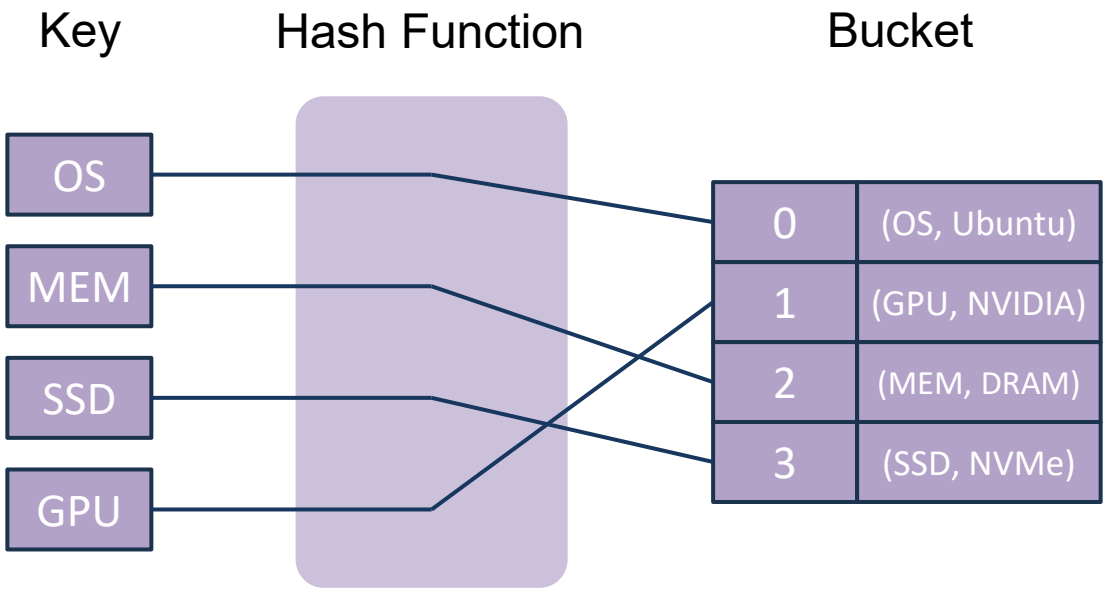
Data Structure

- HashTable
  - How it works
    - Insert
      - Compute hash → find bucket
      - If the key already exists, update its value
      - Otherwise, add a new node to the linked list in that bucket
    - Delete (Remove)
      - Compute hash → find bucket
      - Traverse the linked list to locate the target node
      - Remote the node from the list and reconnect the links
    - Search (Lookup)
      - Compute hash → find bucket
      - Traverse the linked list in that bucket
      - Compare keys until the target key is found

Data Structure

- HashTable – Insert

  - Table Size: 4
  - 주어진 Key에 대해 Hash Function을 이용해 Hash 값 계산
  - 모듈러 연산을 통해 Bucket Index 결정
  - 해당 Bucket에 Value 저장

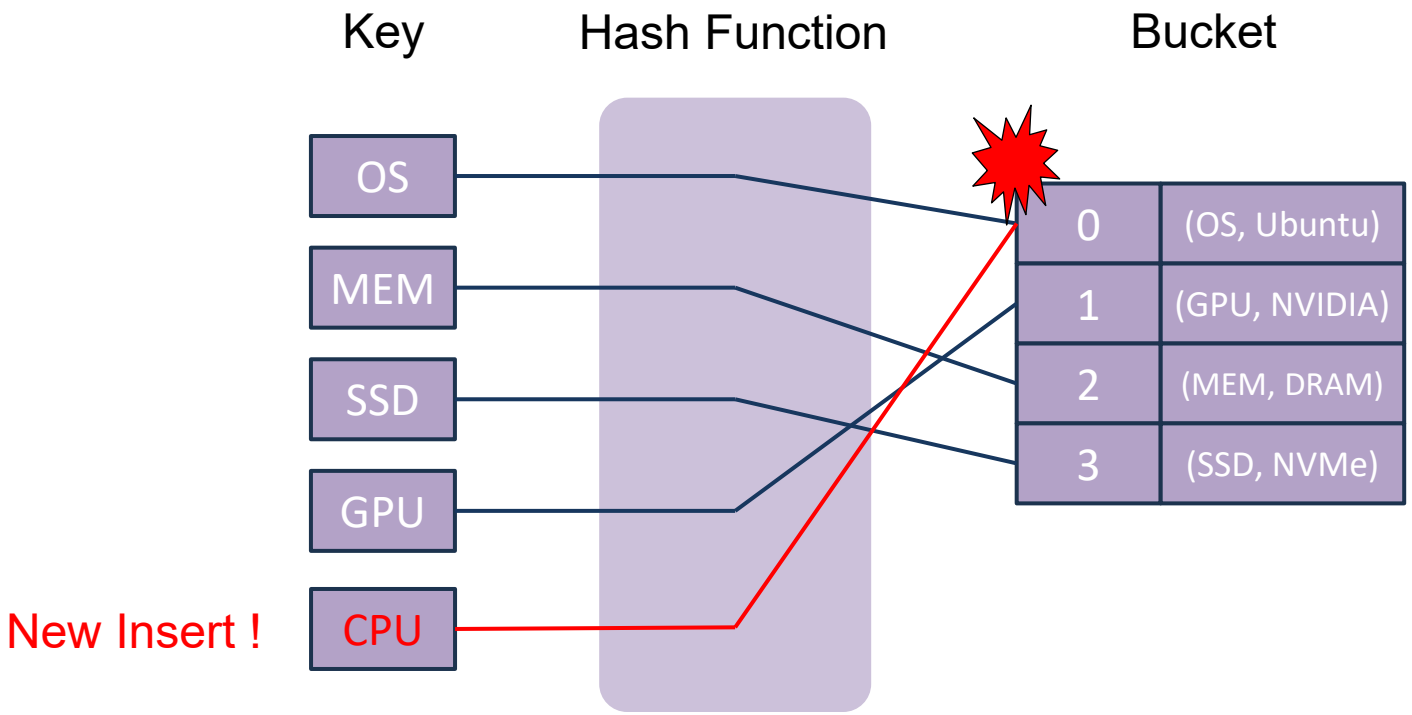


Data Structure

- HashTable – Insert (Hash Collision)

  - Table Size: 4

  - Hash Collision 발생 시, Chaining 기법으로 해결



Hash(CPU) mod Table Size = 0
(CPU, Intel) 의 Bucket Index = 0
하지만 Bucket 0 에는 이미 다른 (K, V) 가 존재
→ Hash Collision

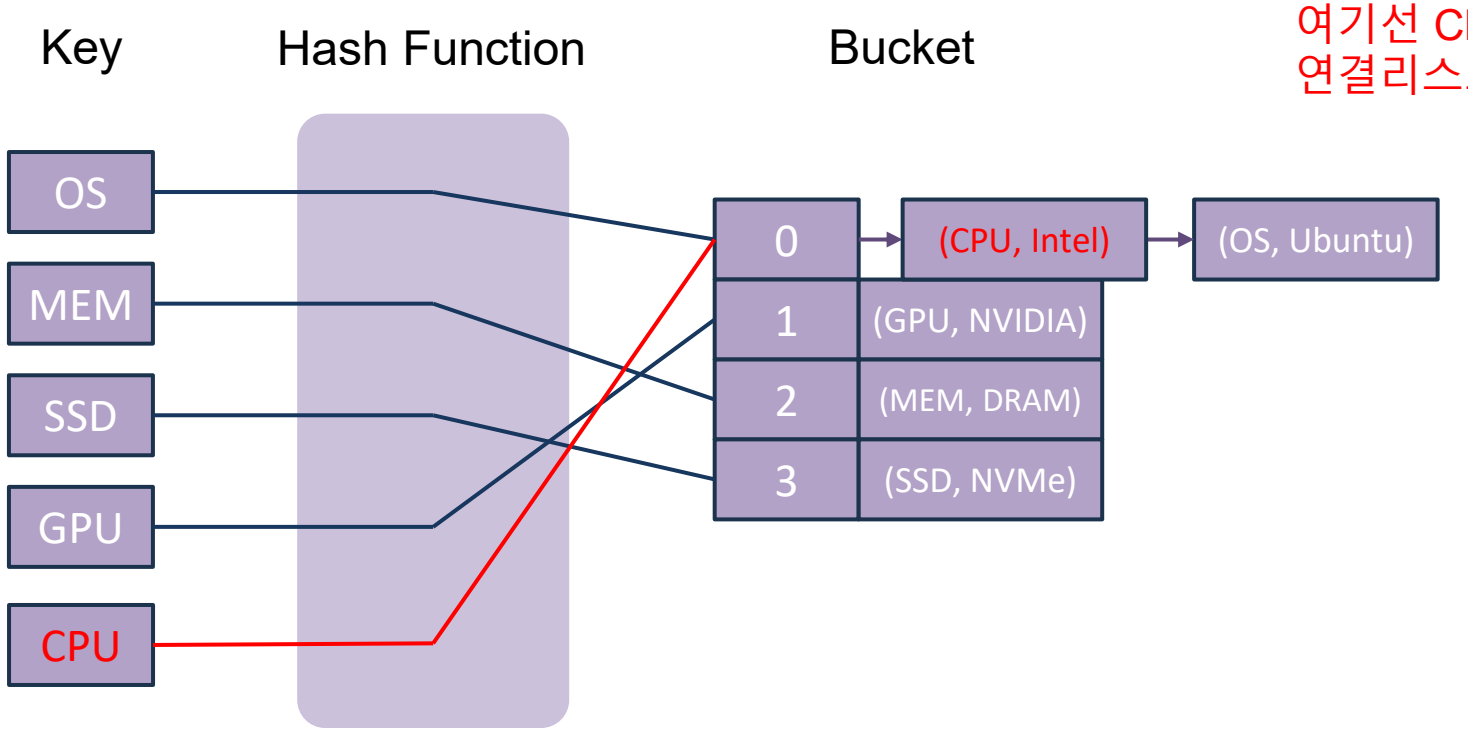
Data Structure

- HashTable – Insert (Hash Collision)

  - Table Size: 4

Hash Collision 기법엔 크게 2가지 존재
- Open Addressing
- Chaining

여기선 Chaining 기법 사용
연결리스트의 맨 앞에 (K, V) Insert 됨

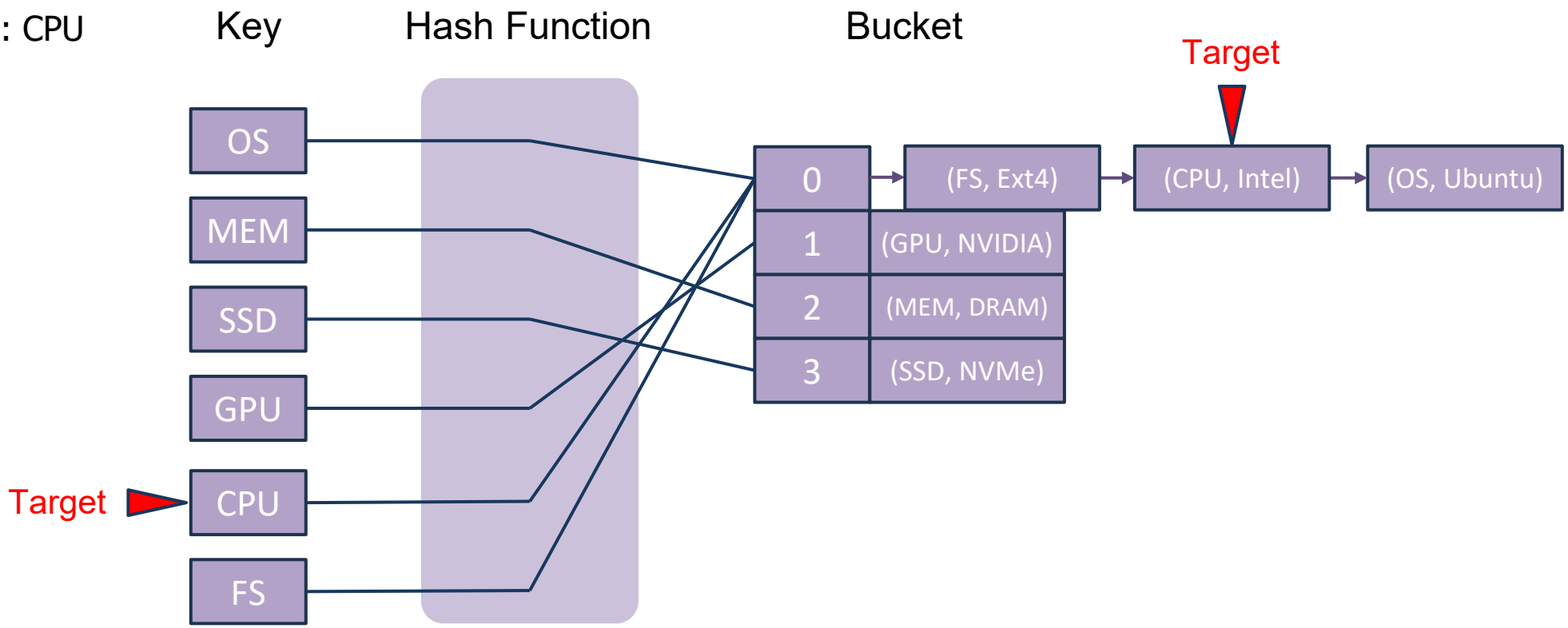


Data Structure

- HashTable – Delete

  - Table Size: 4

  - Target: CPU

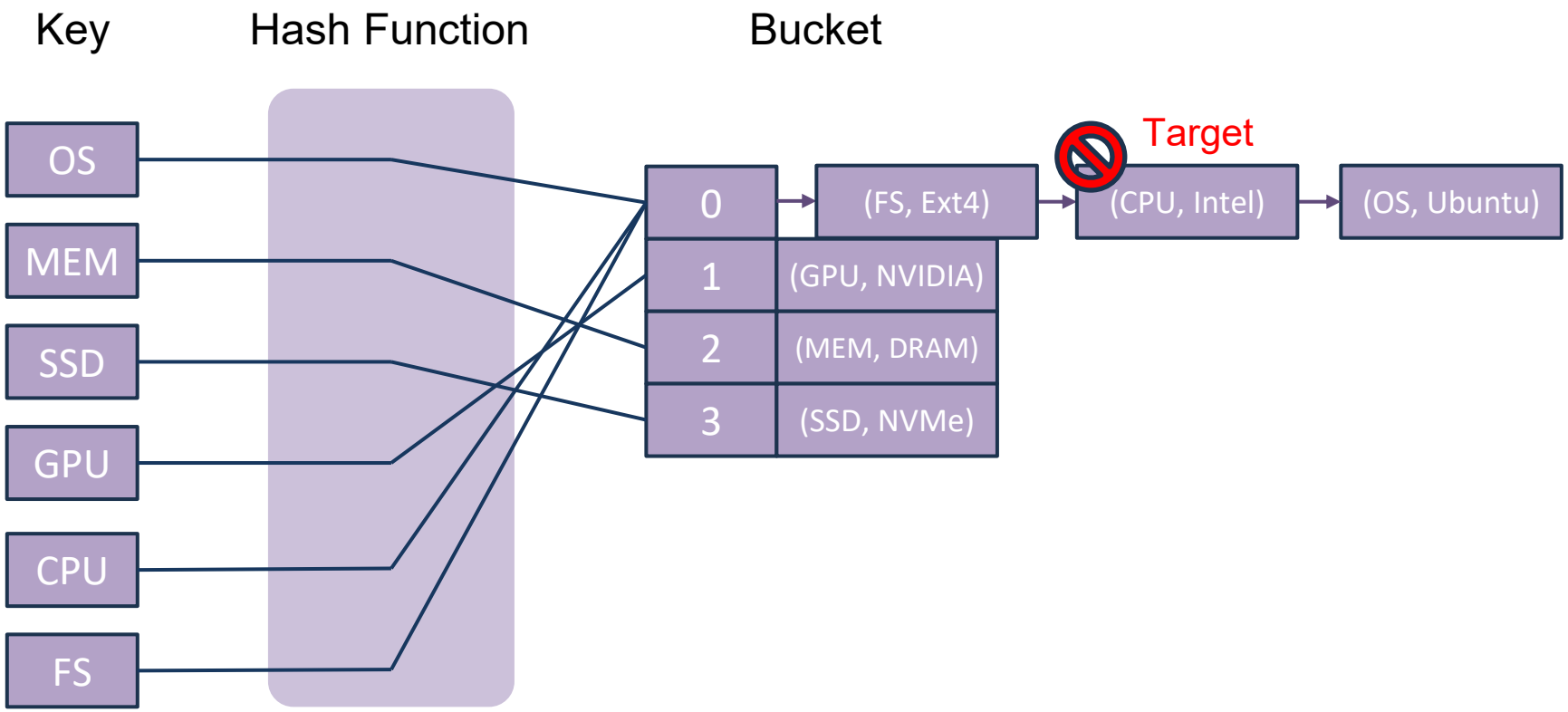


Data Structure

- HashTable – Delete

  - Table Size: 4

  - Target: CPU

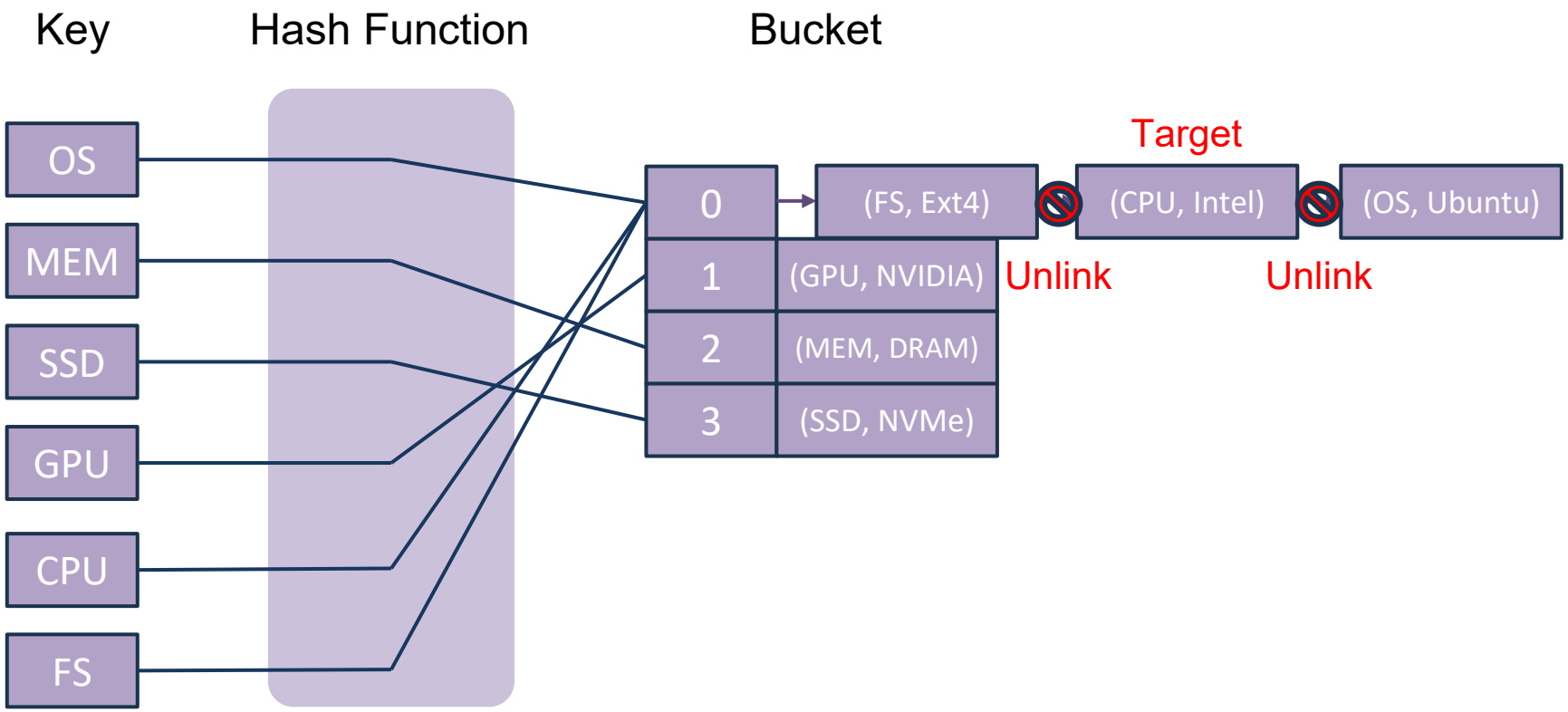


Data Structure

- HashTable – Delete

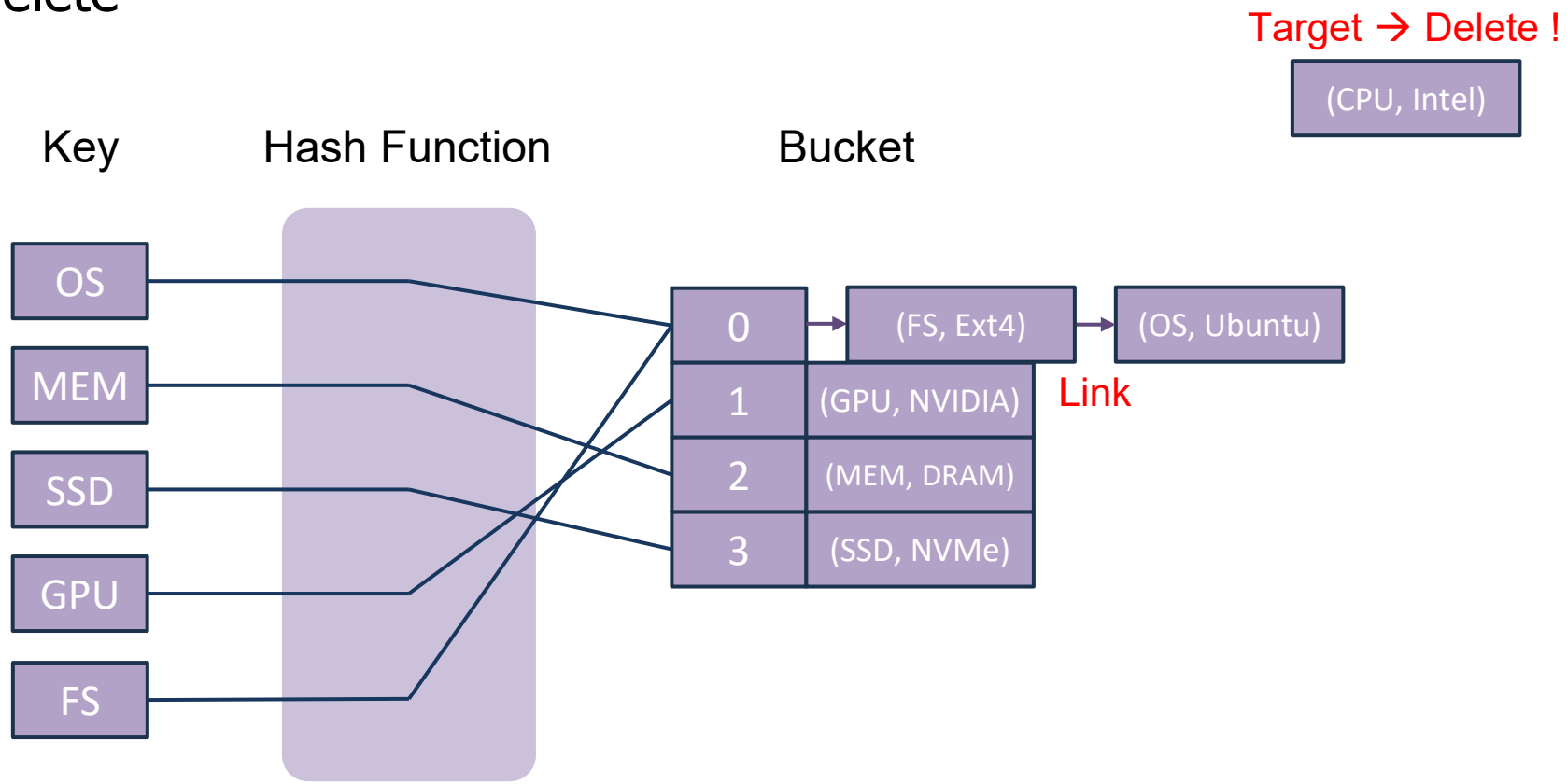
  - Table Size: 4

  - Target: CPU



Data Structure

- HashTable – Delete
  - Table Size: 4
  - Target: CPU

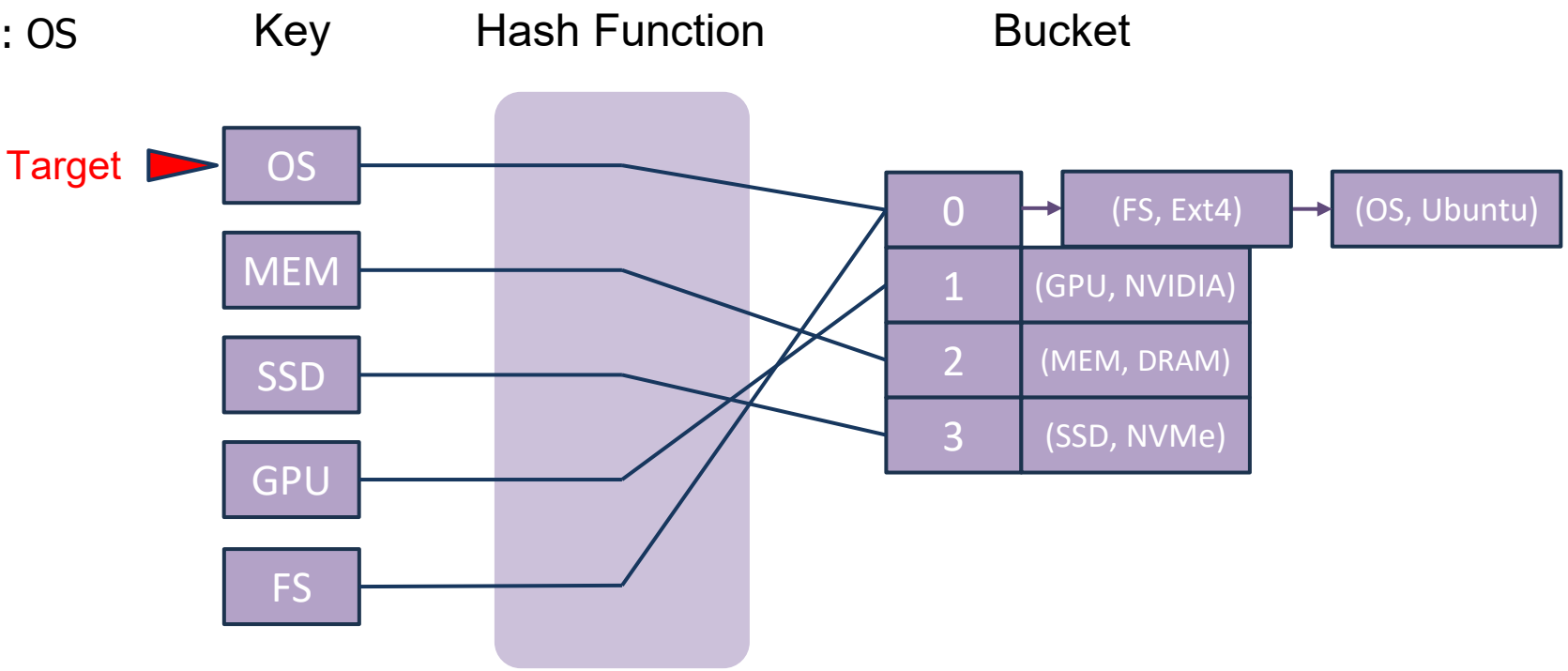


Data Structure

- HashTable – Lookup

  - Table Size: 4

  - Target: OS

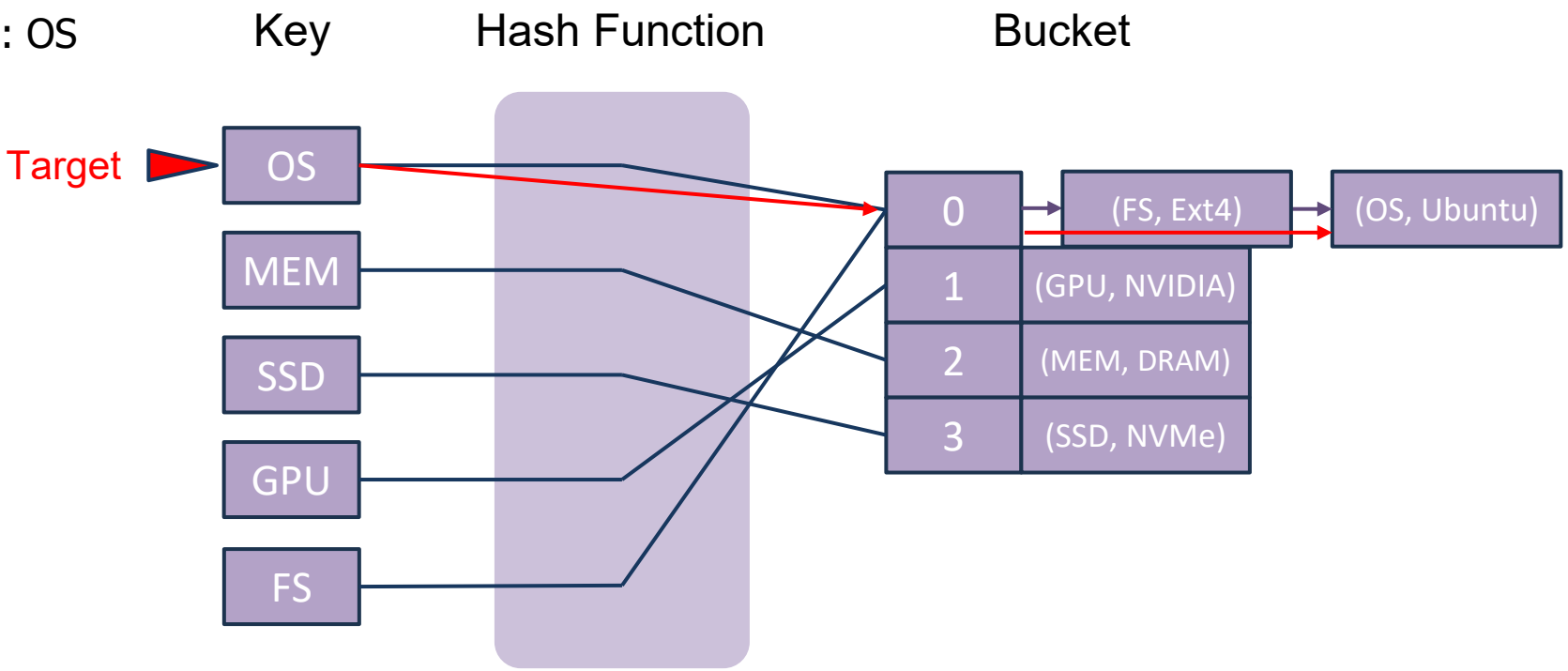


Data Structure

- HashTable – Lookup

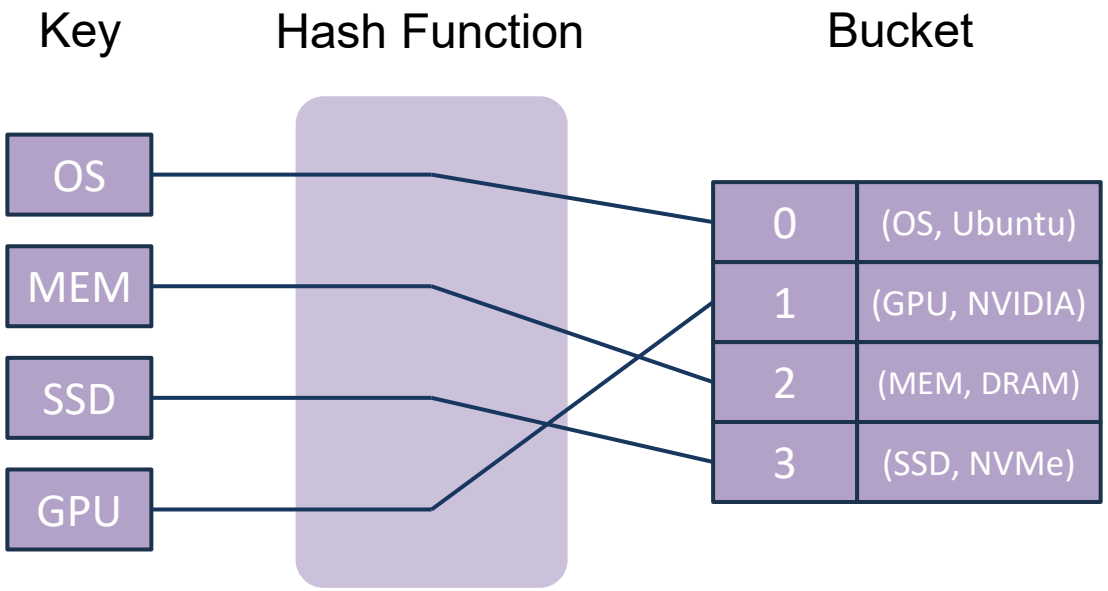
  - Table Size: 4

  - Target: OS



Data Structure

- Lock 구현
  - HashTable 을 다음 3가지 버전으로 구현
    - Without Lock
    - Coarse-grained Lock
    - Fine-grained Lock



Environment Setting

- https://github.com/DKU-EmbeddedSystem-Lab/2026_DKU_OS/
- Ubuntu 사용 (타 OS 사용 X)
- Linux Install Manual 을 보고 환경 설정 권장
- Lab 2 환경 구성
  - $ git clone https://github.com/DKU-EmbeddedSystem-Lab/2026_DKU_OS/
  - $ cd 2026_DKU_OS
  - $ ./install_package.sh
    - 과제에 필요한 패키지 설치 (Vim, g++, OpenSSL, Gtest, …)
  - $ cd lab2
  - $ make clean
  - $ make
  - ./test

소스코드 빌드 및 실행

코드 수정 후 실행할 때 마다 재 빌드 해줘야 함

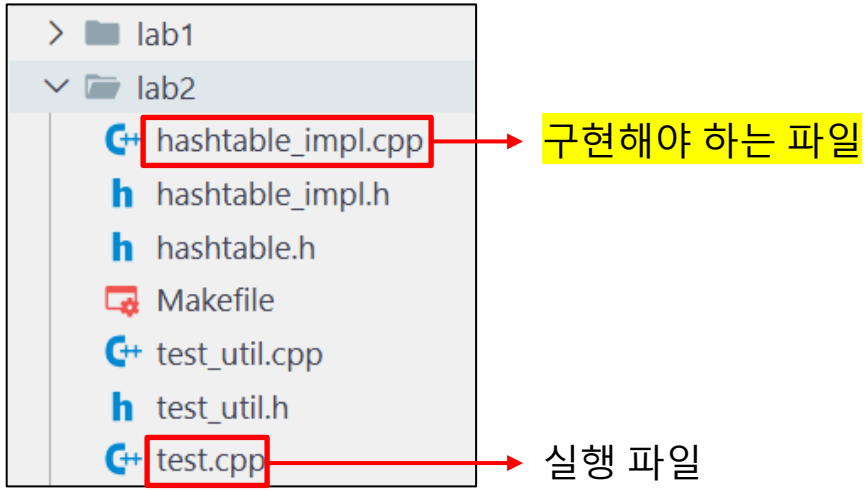
Workload Types

- 유저 Request 종류
  - Insert Only
  - Insert + Lookup (50:50)
  - Insert + Lookup + Delete (60:20:20)

- Key & Value
  - 1,000,000 개
  - 1 ~ 100,000 사이의 랜덤 값

- Thread 개수
  - 1, 2, 4, 8 (Without Lock은 스레드 1개만 수행)

Code Flow



<test.cpp>: main 함수 실행

```
int main(int argc, char** argv) {
    ::testing::InitGoogleTest(&argc, argv);
    return RUN_ALL_TESTS();
}
```

<test.cpp>: single/multi thread, 워크로드에 따른 테스트 케이스 생성

```
/// @brief HashTableSingleThreadTest Test Case and Parameters
INSTANTIATE_TEST_CASE_P(Default, HashTableSingleThreadTest,
    ::testing::Values(
        // Workload Type, Request Num, Num of Threads
        std::make_tuple(INSERT_ONLY,          1000000, 1),
        std::make_tuple(INSERT_LOOKUP,        1000000, 1),
        std::make_tuple(INSERT_LOOKUP_DELETE, 1000000, 1)
    )
);
```

<test_util.cpp>: 워크로드 생성

```
/// @brief TEST_P 실행 전, 실행 됨
void HashTableTest::SetUp() {
    load_workload();
}
```

<test.cpp>: HashTable 객체 생성

```
/// @brief Test HashTable without lock in single-thread
TEST_P(HashTableSingleThreadTest, NoLock) {
    hashtable = new HashTable();
}
```

<test_util.cpp>: 실행 및 결과 확인

```
/// @brief TEST_P 실행 후, 실행 됨
void HashTableTest::TearDown() {
    run_workload();
    check_answer();
}
```

HashTable Class

<hashtable.h>: 노드 구조체

```
/// @brief Hash Table 노드 구조체 (Separate Chaining)
struct HTNode {
    // 키
    int key;
    // 밸류
    int value;
    // 업데이트 횟수
    int upd_cnt;
    // 체이닝: 같은 버킷 내 다음 노드 포인터
    HTNode* next;
};
```

<hashtable.h>: DefaultHashTable 클래스

```
/// @brief HashTable 부모 클래스.
class DefaultHashTable {
    protected:
        // 버킷 개수
        int num_buckets_;
        // 버킷 배열 (각 버킷은 HTNode 체인의 헤드)
        HTNode** buckets_;

    public:
        DefaultHashTable(int num_buckets = 1024);
        virtual ~DefaultHashTable();
```

<hashtable.h>: DefaultHashTable의 멤버 함수
이 함수들을 상속받아 구현하는 것이 과제

```
    /**
     * @brief 삽입 함수 : 인자로 받은 key와 value를 저장한다.
     * 이미 key가 존재하는 경우, 기존 value에 새로운 value를 더하고 update count를 1 증가시킨다.
     *
     * @param key
     * @param value
     */
    virtual void insert(int key, int value) = 0;

    /**
     * @brief 탐색 함수 : 인자로 받은 key의 value를 반환한다.
     * HashTable에 key가 존재하지 않을 경우, 0을 반환한다.
     *
     * @param key
     * @return int
     */
    virtual int lookup(int key) = 0;

    /**
     * @brief 삭제 함수 : 인자로 받은 key가 저장된 노드를 삭제한다.
     *
     * @param key
     */
    virtual void remove(int key) = 0;

    /**
     * @brief 모든 노드를 key 오름차순으로 정렬하여 배열에 저장한다.
     * Hash Table은 정렬된 순서를 보장하지 않으므로, 순회 후 정렬을 수행한다.
     *
     * @param arr
     */
    virtual void traversal(KVC* arr);

protected:
    /**
     * @brief 키에 대한 버킷 인덱스를 계산하는 해시 함수
     *
     * @param key
     * @return int
     */
    int hash_func(int key);
```

hashtable_impl.h

- 부모 클래스를 상속받아 사용
  - 필요시 멤버 변수 / 함수 추가 가능

```
class HashTable : public DefaultHashTable {
    private:
        // 멤버 변수 추가 선언 가능

    public:
        // 생성자
        HashTable(int num_buckets = 1024);
        // 소멸자
        ~HashTable();

        // 멤버 함수 추가 선언 가능
        void insert(int key, int value) override;
        int lookup(int key) override;
        void remove(int key) override;
        void traversal(KVC* arr);
};
```

```
class CoarseHashTable : public DefaultHashTable {
    private:
        // 테이블 전체를 보호하는 단일 뮤텍스
        pthread_mutex_t mutex_lock;

    public:
        // 생성자
        CoarseHashTable(int num_buckets = 1024);
        // 소멸자
        ~CoarseHashTable();

        // 멤버 함수 추가 선언 가능
        void insert(int key, int value) override;
        int lookup(int key) override;
        void remove(int key) override;
        void traversal(KVC* arr);
};
```

```
class FineHashTable : public DefaultHashTable {
    private:
        // 버킷별 뮤텍스 배열 (bucket i 는 bucket_locks[i]로 보호)
        pthread_mutex_t* bucket_locks;

    public:
        // 생성자
        FineHashTable(int num_buckets = 1024);
        // 소멸자
        ~FineHashTable();

        // 멤버 함수 추가 선언 가능
        void insert(int key, int value) override;
        int lookup(int key) override;
        void remove(int key) override;
        void traversal(KVC* arr);
};
```

hashtable_impl.cpp (Implementation)

```
/*
*  DKU Operating System Lab (2026)
*      Lab2 (Concurrency Data Structure: Hash Table)
*      Student id :
*      Student name :
*      Date:
*/
```

필수 !
상단에 인적정보 기입

- Common
  - 생성자, 소멸자, 해시, 순회 함수 제공

```
// DefaultHashTable 생성자
DefaultHashTable::DefaultHashTable(int num_buckets) : num_buckets_(num_buckets) {
    // 버킷 배열 할당 및 초기화 (모두 nullptr)
    buckets_ = new HTNode*[num_buckets_];
    for (int i = 0; i < num_buckets_; i++) {
        buckets_[i] = nullptr;
    }
}
```

```
// DefaultHashTable 소멸자
DefaultHashTable::~DefaultHashTable() {
    // 각 버킷의 체인 노드 전부 해제
    for (int i = 0; i < num_buckets_; i++) {
        HTNode* node = buckets_[i];
        while (node) {
            HTNode* tmp = node->next;
            delete node;
            node = tmp;
        }
    }
    delete[] buckets_;
}
```

```
// 해시 함수 : key를 버킷 수로 나눈 나머지
int DefaultHashTable::hash_func(int key) {
    return key % num_buckets_;
}
```

```
// 순회 함수 : 전체 버킷을 순회하여 key 오름차순으로 정렬 후 arr에 저장
// Hash Table은 삽입 순서를 보장하지 않으므로, std::sort로 정렬한다.
void DefaultHashTable::traversal(KVC* arr) {
    std::vector<KVC> tmp;
    for (int i = 0; i < num_buckets_; i++) {
        HTNode* node = buckets_[i];
        while (node) {
            KVC kvc;
            kvc.key     = node->key;
            kvc.value   = node->value;
            kvc.upd_cnt = node->upd_cnt;
            tmp.push_back(kvc);
            node = node->next;
        }
    }
    // key 기준 오름차순 정렬 (std::map 순회와 동일한 순서 보장)
    std::sort(tmp.begin(), tmp.end(), [](const KVC& a, const KVC& b) {
        return a.key < b.key;
    });
    for (int i = 0; i < (int)tmp.size(); i++) {
        arr[i] = tmp[i];
    }
}
```

hashtable_impl.cpp (Implementation)

```
/*
*  DKU Operating System Lab (2026)
*      Lab2 (Concurrency Data Structure: Hash Table)
*      Student id :
*      Student name :
*      Date:
*/
```

필수 !
상단에 인적정보 기입

- HashTable
  - Without Lock 버전 (Single Thread 전용)

```
// HashTable 생성자
HashTable::HashTable(int num_buckets) : DefaultHashTable(num_buckets) {}

// HashTable 소멸자
HashTable::~HashTable() {}

void HashTable::insert(int key, int value) {
    // 구현
}

int HashTable::lookup(int key) {
    // 구현
}

void HashTable::remove(int key) {
    // 구현
}

void HashTable::traversal(KVC* arr) {
    // 구현
}
```

구현해야 하는 부분

hashtable_impl.cpp (Implementation)

```
/*
*  DKU Operating System Lab (2026)
*      Lab2 (Concurrency Data Structure: Hash Table)
*      Student id :
*      Student name :
*      Date:
*/
```

필수 !
상단에 인적정보 기입

- CoarseHashTable
  - Coarse-grained Lock 버전

```
// CoarseHashTable 생성자
CoarseHashTable::CoarseHashTable(int num_buckets) : DefaultHashTable(num_buckets) {
    pthread_mutex_init(&mutex_lock, nullptr);
}

// CoarseHashTable 소멸자
CoarseHashTable::~CoarseHashTable() {
    pthread_mutex_destroy(&mutex_lock);
}

void CoarseHashTable::insert(int key, int value) {
    // 구현
}

int CoarseHashTable::lookup(int key) {
    // 구현
}

void CoarseHashTable::remove(int key) {
    // 구현
}

void CoarseHashTable::traversal(KVC* arr) {
    // 구현
}
```

구현해야 하는 부분

hashtable_impl.cpp (Implementation)

```
/*
*  DKU Operating System Lab (2026)
*      Lab2 (Concurrency Data Structure: Hash Table)
*      Student id :
*      Student name :
*      Date:
*/
```

필수 !
상단에 인적정보 기입

- FineHashTable
  - Fine-grained Lock 버전
  - 생성자 및 소멸자 새로 정의

```cpp
// FineHashTable 생성자
FineHashTable::FineHashTable(int num_buckets) : DefaultHashTable(num_buckets) {
    // 버킷 수만큼 뮤텍스 배열 할당 및 초기화
    bucket_locks = new pthread_mutex_t[num_buckets_];
    for (int i = 0; i < num_buckets_; i++) {
        pthread_mutex_init(&bucket_locks[i], nullptr);
    }
}
```

```cpp
// FineHashTable 소멸자
FineHashTable::~FineHashTable() {
    for (int i = 0; i < num_buckets_; i++) {
        pthread_mutex_destroy(&bucket_locks[i]);
    }
    delete[] bucket_locks;
}
```

```cpp
void FineHashTable::insert(int key, int value) {
    // 구현
}

int FineHashTable::lookup(int key) {
    // 구현
}

void FineHashTable::remove(int key) {
    // 구현
}

void FineHashTable::traversal(KVC* arr) {
    // 구현
}
```

→ 구현해야 하는 부분

Result

- 프로그램 실행 결과

```
[==========] Running 27 tests from 2 test suites.
[----------] Global test environment set-up.
[----------] 9 tests from Default/HashTableSingleThreadTest
[ RUN      ] Default/HashTableSingleThreadTest.NoLock/0
Execution time: 607 ms
[       OK ] Default/HashTableSingleThreadTest.NoLock/0 (1249 ms)
[ RUN      ] Default/HashTableSingleThreadTest.NoLock/1
Execution time: 584 ms
[       OK ] Default/HashTableSingleThreadTest.NoLock/1 (1031 ms)
[ RUN      ] Default/HashTableSingleThreadTest.NoLock/2
Execution time: 585 ms
[       OK ] Default/HashTableSingleThreadTest.NoLock/2 (1219 ms)
[ RUN      ] Default/HashTableSingleThreadTest.Coarse/0
Execution time: 632 ms
[       OK ] Default/HashTableSingleThreadTest.Coarse/0 (1252 ms)
[ RUN      ] Default/HashTableSingleThreadTest.Coarse/1
Execution time: 608 ms
[       OK ] Default/HashTableSingleThreadTest.Coarse/1 (1036 ms)
```

Project Requirements

- 요구사항

  - 본 과제에서는 주어진 Lock 적용 범위에 맞춰 HashTable을 구현하는 것을 목표로 한다

    - Without Lock, Coarse-grained Lock, Fine-grained Lock

  - HashTable은 유저의 요청에 따라 Insert, Lookup, Remove 연산을 수행한다

    - Insert: 인자로 받은 Key의 Hash 값을 기반으로 Bucket Index 계산하여 Value 를 저장한다.

      Bucket Index 에 이미 Value 가 있다면 (Hash Collision) Chaining 기법으로 해결한다.

      동일 Key에 대해 Insert 요청이 들어오면 기존 Value에 새 Value 를 더하고, upd_cnt 를 1씩 증가시킨다.

    - Lookup: 인자로 받은 Key의 Value를 반환하다.

      Key가 존재하지 않을 경우 0을 반환한다.

    - Remove: 인자로 받은 Key가 있는 Bucket 에서 일치하는 Node 를 삭제한다.

Workload Format

- 입력 (Workload)
  - Key 의 개수: 1,000,000 개
  - Key, Value 의 범위: 1 <= key, value <= 100,000
  - 첫 번째 요청은 항상 Insert {key=1, Value=1} 로 설정된다
  - Key, Value 는 랜덤한 값으로 생성된다
  - 1, 2, 4, 8 개의 스레드를 사용하여 테스트 진행 (Without Lock 버전은 1개의 스레드만 사용)

Implementation Details

- 구현해야 하는 내용
  - hashtable.cpp 에서 HashTable, CoarseHashTable, FineHashTable 클래스를 구현한다
    - 각 클래스의 생성자, 소멸자, 해시, 순회 함수는 기본적으로 제공된다
    - 각 클래스에서 1) Insert, 2) Lookup, 3) Remove 함수를 구현해야 한다
      - 클래스에 적용되는 Lock의 범위에 맞춰 구현을 진행해야 한다
    - 반드시 C++로 구현해야 하며 라이브러리는 C++ STL만 사용 가능하다
    - hashtable_impl.cpp, hashtable_impl.h 외 다른 파일은 수정, 추가, 제출이 불가하다

Code Submission

- 양식
  - 제목: os_lab2_분반_학번_이름.cpp
    - Ex) os_lab2_2_32222797_위다연.cpp + os_lab2_32222797_위다연.h
  - 코드 상단에 작성자 정보 기입 및 코드 설명하는 주석 달기 (line by line)
  - Make 로 컴파일이 되고, 정상적으로 실행이 되어야 함
  - C++ 로 작성
  - Fine-grained Lock의 성능은 Coarse-grained Lock 보다 우수해야 함
    - 단, 특정 실행 환경에서만 성능이 향상된 경우는 인정하지 않으며, 조교의 평가 환경에서도 동일하게 성능 향상이 확인되어야 함
  - 주석을 생성형 AI (Gemini, ChatGPT 등) 캡쳐 또는 그대로 복사.붙여넣기한 경우, 기본점수 처리
    - 생성형 AI의 도움을 받는 것은 가능하나, 코드의 동작 원리를 충분히 이해하고 있어야 함

Report Guidelines

- 보고서 내용
  - 구현한 소스코드 설명
  - 문제 풀이
    - 멀티 스레드 환경에서 HashTable에 접근할 때, 요청 수행 순서를 일관되게 보장하는 방법을 설명하라
    - Fine-grained HashTable에서는 동일한 Bucket에 대해 서로 다른 스레드에서 동시에 Insert와 Remove를 요청할 경우, 요청의 처리 순서에 따라 결과가 어떻게 달라질 수 있는지 설명하라
  - Discussion
    - 여러 워크로드에서 Lock이 적용되는 범위가 다를 경우, Thread 의 개수에 따른 실행시간 비교 분석
    - 과제를 진행하며 새롭게 배운 점 / 어려웠던 점
    - 기타 내용 자유롭게 작성
  - 생성형 AI (Gemini, ChatGPT) 캡쳐 또는 그대로 복붙의 경우, 보고서 점수 0점 처리

Report Submission

- 양식
  - 제목: os_lab2_분반_학번_이름.pdf
    - Ex) os_lab2_2_32222797_위다연.pdf
    - 구글폼 제출 시 파일명 뒤에 본인의 구글 닉네임이 추가됨 (감점 X)
    - 구글폼 재제출 가능 (가장 마지막에 제출한 제출물을 기준으로 채점)
  - 코드 및 터미널 화면 첨부 시 흰색 바탕으로 캡쳐
    - 그렇지 않을 시, 감점의 요인이 됨
    - VS code 사용자: VS Code 테마 변경 방법
    - Linux 터미널: 리눅스 터미널 색상 변경 방법

Grading Criteria

- 구현
  - 실행 결과가 정답과 일치하는가? (모든 test 결과가 OK인지 확인)
  - 스레드 수 증가에 따라 Fine-grained 의 실행 시간이 Coarse-grained 의 실행 시간보다 빠른가?
    - Fine-grained 의 성능이 Coarse-grained 보다 우수해야 함
  - 소스코드에 각 줄(or 코드 블록)마다 주석이 잘 작성되어 있는가?
  - Make를 통해 컴파일 및 실행이 정상적으로 동작하는가?
    - 별도의 컴파일 방법 또는 실행 방법을 보고서에 서술해도 인정되지 않음
- 보고서
  - 각 스케줄링 기법의 성능을 그래프 등의 지표를 활용하여 비교/분석하였는가?
  - 구현한 소스코드에 대한 설명이 충분히 잘 작성되어 있는가?

Grading Criteria

- 총점 100점 (자세한 배점은 우측 표 참고)
  - Without Lock 만 구현한 경우 (10점)
  - Coarse-grained Lock 까지 구현한 경우 (10점 + 20점)
  - Fine-grained Lock 까지 구현한 경우 (10점 + 20점 + 30점)
  - 보고서 30점 + 양식 10점

구분	세부사항	점수
구현	Without Lock	10
	Coarse-grained Lock	20
	Fine-grained Lock	30
보고서	구현 내용 설명	20
	Discussion	10
양식	주어진 양식 준수	10

- 유의사항
  - 지각 제출 시, 하루에 30% 감점
  - 소스코드 제출 기준 미 준수 시 → 구현 0점 + 형식 점수 0점

    ex) 소스코드 텍스트로 제출, make 실행 안됨, …
  - 인적사항 주석 미 작성, 파일명 / 형식 미 준수 시 → 형식 점수 0점

Submission Guideline

기한: **2026.05.13 (수) 23:59**까지

- 제출

  - 1분반: https://docs.google.com/forms/d/e/1FAIpQLSdw8LWyc0cmNBUksXF-0NY-dExDHqi_srbZXpIoZuDGGFI_nA/viewform?usp=header

  - 2분반: https://docs.google.com/forms/d/e/1FAIpQLSeXGjIOapNwTa3mwGnwTdvNOYtlIIbWgsF-SuizxGTFE7wTCw/viewform?usp=header

  - 3분반: https://docs.google.com/forms/d/e/1FAIpQLScPxDw6rQKl-lwNcjlOT2ftnuFA9xsQGrz_CPjsliFZ4h-S-g/viewform?usp=header

  - 구글 폼 양식에 맞춰 제출 (재제출 가능, 가장 마지막 제출을 기준으로 채점)
  - 본인 분반에 맞게 제출, 그렇지 않을 시 미제출로 간주함
  - 이메일로 제출 금지, 이메일로 제출 시 회신 없이 미제출로 간주함
  - 구글 폼 제출 시 본인의 메일로 사본을 받을 수 있음

    → 따라서 본인이 코드 잘못 제출 시, 책임은 본인에게 있으며 미제출로 간주될 수 있음

  - **질문이 있을 시 조교의 이메일 또는 git issue에 업로드**
  - **메일 작성 시, 본인의 분반과 이름 작성 바랍니다.**

Thank You

2026.04.29

T.A. 위다연

weedayeon@dankook.ac.kr